Comparative Evaluation of Traditional Machine Learning and Deep Learning Models for DNA Sequence Classification

Abstract

DNA sequence classification is an important bioinformatics task for organizing biological sequences into predefined categories. This study compares four machine-learning approaches for classifying DNA sequences into seven gene-family classes: Multinomial Naive Bayes, Support Vector Machine (SVM), XGBoost, and a one-dimensional Convolutional Neural Network (1D CNN). The experiments use the Human, Chimpanzee, and Dog files from the Kaggle DNA Sequence Dataset. After cleaning and combining records, the dataset contains 6,882 sequences. The traditional classifiers use k-mer count features, with six-mers used for the main comparison. The CNN uses one-hot-encoded sequences padded to a maximum length of 4,295 bases. A stratified 80:20 train-test split produces 5,505 training and 1,377 test records. SVM achieves the highest test accuracy (94.34%) and Macro F1-score (94.46%). XGBoost achieves the highest Macro ROC-AUC (99.64%), while the 1D CNN achieves 86.35% accuracy and 85.27% Macro F1. Multinomial Naive Bayes provides a baseline accuracy of 65.50%. In an SVM k-mer experiment, six-mers outperform k values of three, four, and five. These results suggest that k-mer-based SVM and XGBoost are effective baselines for this dataset. The findings are limited to the selected dataset and split; additional validation, external testing, and biological interpretation are needed before broader conclusions can be drawn.

Keywords: DNA sequence classification, bioinformatics, k-mer, Support Vector Machine, XGBoost, convolutional neural network, multi-class classification.

1.INTRODUCTION

1.1 Background and Motivation

DNA sequence classification is an important task in bioinformatics. DNA sequences contain biological information that helps researchers study genes and their functions. Classifying DNA sequences into different gene families helps organize biological data and identify patterns in genetic sequences. Machine-learning techniques can be used to classify DNA sequences based on their sequence patterns.

Different machine-learning models may perform differently on the same classification task. Traditional machine-learning models can use k-mer features extracted from DNA sequences, while deep-learning models can learn sequence patterns from encoded data. Therefore, comparing different models is useful for understanding their performance in DNA sequence classification.

This study compares four machine-learning models: Multinomial Naive Bayes, Support Vector Machine (SVM), XGBoost, and a one-dimensional Convolutional Neural Network (1D CNN). The motivation is to evaluate these models using the same dataset and identify which model performs best for classifying DNA sequences into seven gene-family classes.

1.2 DNA Sequencing and Machine Learning

DNA is composed of four nucleotide bases: adenine (A), thymine (T), cytosine (C), and guanine (G). A DNA sequence is represented as a string of these bases. Machine-learning techniques can use patterns in these sequences to classify them into predefined categories.

One common approach is k-mer representation, which divides a DNA sequence into overlapping subsequences of length \(k\). In this study, k-mer count features are used with Multinomial Naive Bayes, SVM, and XGBoost. The main comparison uses six-mers, and additional experiments examine k-mer sizes of three, four, five, and six for SVM.

Deep learning provides another approach to sequence classification. This study uses a one-dimensional Convolutional Neural Network (1D CNN) with one-hot-encoded DNA sequences. The experiments use the Human, Chimpanzee, and Dog files from the selected Kaggle DNA Sequence Dataset. After cleaning and combining the records, 6,882 sequences are used to classify seven gene-family classes: G protein-coupled receptor, tyrosine kinase, tyrosine phosphatase, synthetase, synthase, ion channel, and transcription factor.
The performance of the four models is evaluated using accuracy, precision, recall, Macro F1-score, and Macro ROC-AUC. This comparison provides an experimental assessment of traditional machine-learning and deep-learning approaches for the selected DNA sequence classification task.

2. RELATED WORK

DNA sequence classification has been studied using traditional machine-learning methods, k-mer representations, and deep-learning architectures. These methods differ in how they represent DNA sequences and learn patterns for classification. The following studies are relevant to the present work.

Gunasekaran et al. (2021) investigated CNN and hybrid deep-learning architectures for DNA sequence classification. Their experiments included CNN, CNN-LSTM, and CNN-BiLSTM models with different sequence encoding approaches. CNN with k-mer encoding achieved 93.16% test accuracy, showing the potential of combining sequence representations with deep-learning models.

Menzel et al.'s BERTax study (2022) introduced a deep-learning approach for taxonomic classification of DNA sequences. The model learns sequence patterns without depending entirely on similarity to known reference sequences. The authors reported performance comparable to existing methods for familiar taxonomic groups and better performance for novel organisms.

Seth et al. (2023) studied gene-family classification using machine learning and k-mer counting. Their approach classified DNA sequences into seven gene families and reported 90.9% accuracy. This work is particularly relevant because it uses k-mer features for gene-family classification, similar to the feature representation used in the present study.

Pu and Shamir (2024) developed 4CAC, a classifier for identifying viruses, plasmids, prokaryotes, and microeukaryotes in metagenomic assemblies. Their method combines sequence-length-adjusted XGBoost models with assembly-graph information. The study demonstrated the usefulness of XGBoost for genomic classification, although its classification task differs from the seven gene-family classes used here.

Chakraborty and Shanthi (2026) proposed an explainable AI framework for multi-species DNA functional-group classification. They used Human, Chimpanzee, and Dog datasets and their combined dataset of 6,882 sequences. Their experiments included six-mer features, an 80:20 stratified split, traditional machine-learning models, and deep-learning architectures. Logistic Regression achieved the highest MCC and F1-scores in their experiments, and the authors also investigated model explanations and biological sequence motifs. This study is closely related to the present work because it uses the same combined dataset.

The reviewed studies show that both k-mer-based machine learning and deep learning can be useful for DNA sequence classification. Building on this work, the present study evaluates Multinomial Naive Bayes, SVM, XGBoost, and 1D CNN, and examines how k-mer size affects SVM performance. The results are specific to the selected dataset and experimental setup.

Table: Summary of Related Research

No.	**Author(s), Year**	**Technique**	**Objective**	**Reported outcome**
1	Gunasekaran et al., 2021	CNN, CNN-LSTM, CNN-BiLSTM; k-mer and label encoding	Compare deep-learning architectures for DNA sequence classification.	CNN with k-mer encoding achieved 93.16% test accuracy.
2	Menzel et al., 2022	BERTax; deep neural network	Classify DNA sequences taxonomically beyond direct sequence similarity.	Comparable results for familiar groups and improved classification of novel organisms.
3	Seth et al., 2023	Machine learning with k-mer counting	Classify DNA sequences into seven gene families.	Reported 90.9% classification accuracy.
4	Pu and Shamir, 2024	XGBoost and assembly-graph information	Classify metagenomic contigs into four biological groups.	Improved classification compared with existing methods, particularly for short reads.

5	Chakraborty and Shanthi, 2026	K-mer-based ML, deep learning, and explainable AI	Classify DNA functional groups across Human, Chimpanzee, and Dog datasets.	Logistic Regression achieved the highest MCC and F1-scores in their experiments; the combined dataset contained 6,882 sequences.

3. METHODOLOGY

A.Dataset Description and Preparation

This study uses the DNA Sequence Dataset available on Kaggle. Three source files, namely Human, Chimpanzee, and Dog, were used to prepare the dataset. The Human file contains 4,380 records, the Chimpanzee file contains 1,682 records, and the Dog file contains 820 records. After cleaning and combining the records, the final dataset contains 6,882 DNA sequences.

Each sequence is associated with one of seven gene-family classes: G protein-coupled receptor, tyrosine kinase, tyrosine phosphatase, synthetase, synthase, ion channel, and transcription factor. These classes form the target labels for the classification task. The objective is to predict the gene-family class of a DNA sequence rather than its species of origin.

The combined dataset was divided into training and testing sets using a stratified 80:20 split with a random state of 42. The training set contains 5,505 sequences, while the testing set contains 1,377 sequences. Stratification was used to preserve the class distribution across the two sets.

B. DNA Sequence Feature Representation

Two feature representation approaches were used to prepare the DNA sequences for classification: k-mer count representation for traditional machine-learning models and one-hot encoding for the 1D CNN.

1.K-mer Count Representation

A k-mer is a subsequence containing (k) consecutive nucleotide bases. In a DNA sequence, overlapping k-mers can be extracted to represent local sequence patterns. For example, a sequence containing the bases A, C, G, and T can be divided into shorter overlapping subsequences according to the selected value of (k).

In this study, CountVectorizer was used to convert DNA sequences into k-mer count feature vectors. Six-mers ((k=6)) were used for the main comparison of Multinomial Naive Bayes, Support Vector Machine (SVM), and XGBoost. This representation provides numerical features that can be used by the traditional classification algorithms.

An additional experiment investigated the effect of k-mer size on SVM performance. The values (k=3), (k=4), (k=5), and (k=6) were evaluated to examine how different sequence lengths affected classification performance.

2. One-Hot Sequence Encoding

For the 1D CNN, DNA sequences were represented using one-hot encoding. In this representation, nucleotide bases are converted into numerical vectors so that the neural network can process the sequences as input.
Because DNA sequences can have different lengths, the sequences were padded to a maximum length of 4,295 bases. This length was selected based on the 95th percentile of the sequence-length distribution. The resulting encoded sequences were used as input to the CNN for learning local patterns associated with the seven gene-family classes.

C. Classification Models

Four classification models were implemented and evaluated: Multinomial Naive Bayes, SVM, XGBoost, and a one-dimensional Convolutional Neural Network. The first three models used k-mer count features, whereas the CNN used one-hot-encoded sequences.

1.Multinomial Naive Bayes

Multinomial Naive Bayes is a probabilistic classification algorithm that applies Bayes' theorem and assumes conditional independence among features given the class. It was used as a baseline model with the six-mer count representation. Its classification performance provides a reference for comparison with the other models.

2.Support Vector Machine

Support Vector Machine is a supervised learning algorithm that identifies a decision boundary for separating classes. A LinearSVC classifier was used with the six-mer count features. The main model used a regularization parameter of (C=1.0), balanced class weights, and a random state of 42.
Two additional SVM experiments were conducted. First, k-mer sizes of 3, 4, 5, and 6 were compared to investigate the effect of feature length. Second, GridSearchCV with three-fold cross-validation was used to examine the regularization parameter (C) over the values 0.1, 1, and 10. The tuning experiment used Macro F1-score to select the best parameter.

3. XGBoost

XGBoost is a gradient-boosting algorithm that builds an ensemble of decision trees. It was used to classify the DNA sequences based on six-mer count features.
The model was configured with 200 estimators, a maximum tree depth of 6, a learning rate of 0.1, a subsample ratio of 0.8, and a column-sampling ratio of 0.8. The objective was set to multiclass classification with seven output classes, and the evaluation metric was set to multiclass log loss. The random state was set to 42, and parallel processing was enabled.
Feature importance was also examined to identify which input features contributed most to the trained XGBoost model's predictions.

4.One-Dimensional Convolutional Neural Network

A one-dimensional Convolutional Neural Network was implemented to classify one-hot-encoded DNA sequences. The architecture consisted of a first Conv1D layer with 64 filters and a kernel size of 7, followed by max pooling. A second Conv1D layer used 128 filters and a kernel size of 5. Global max pooling was then applied to obtain a fixed-length representation.

The resulting representation was passed to a dense layer containing 64 units, followed by a dropout layer with a rate of 0.4. The final dense layer contained seven output units with a softmax activation function to produce class probabilities for the seven gene-family classes.

The model was trained for a maximum of 10 epochs using a batch size of 32. Early stopping was used during training to help prevent unnecessary training when the monitored performance stopped improving.

D. Experimental Design and Model Training

The experimental design consisted of a main model comparison and two additional SVM experiments. In the main comparison, Multinomial Naive Bayes, SVM, and XGBoost were trained using six-mer count features, while the 1D CNN was trained using one-hot-encoded DNA sequences.

All four models were evaluated using the same stratified training and testing split. The test set contained 1,377 sequences and was used to compare the classification performance of the trained models.

The first additional experiment investigated the effect of k-mer size on SVM performance. Four feature configurations were evaluated using (k=3), (k=4), (k=5), and (k=6). The second experiment used GridSearchCV with three-fold cross-validation to tune the SVM regularization parameter among (C=0.1), (C=1), and (C=10).

For the CNN, training was conducted for up to 10 epochs with a batch size of 32 and early stopping. Training time was also recorded for the traditional machine-learning models to support a comparison of computational cost.

The main comparison used one fixed test set. Therefore, the reported results represent performance on this particular split and should not be interpreted as a guarantee of performance on independent datasets.

E. Evaluation Metrics

The performance of the four classification models was evaluated using accuracy, macro-averaged precision, macro-averaged recall, Macro F1-score, and Macro ROC-AUC. These metrics provide different views of classification performance across the seven gene-family classes.

Accuracy: Accuracy measures the proportion of all test sequences that were classified correctly.

Accuracy = Correct Predictions / Total Predictions

Macro Precision: Precision measures the proportion of correct positive predictions among all positive predictions for a class. Macro precision is calculated by computing precision for each class and taking the unweighted average across the seven classes.

Macro Recall: Recall measures the proportion of actual samples of a class that were correctly identified. Macro recall is calculated by averaging the recall values of all seven classes equally.

Macro F1-score: The F1-score is the harmonic mean of precision and recall. Macro F1-score is obtained by calculating the F1-score for each class and then taking their unweighted average.

Macro ROC-AUC: The Receiver Operating Characteristic Area Under the Curve (ROC-AUC) measures the ability of a model to distinguish between classes. For this multiclass task, Macro ROC-

AUC is calculated by evaluating each class against the remaining classes and averaging the resulting AUC values equally.
Macro-averaged metrics are useful for this study because they give equal weight to each gene-family class rather than allowing classes with more sequences to dominate the overall score.

F. Overall Classification Workflow

The complete workflow begins with collecting the three source files and preparing the combined dataset. The data is then divided into training and testing sets using a stratified split. Two feature representation pipelines are used: k-mer count features for the traditional machine-learning models and one-hot encoding for the 1D CNN.
The four models are trained using their respective input representations and evaluated on the test set. Additional experiments investigate the effect of k-mer size and SVM hyperparameter tuning. Finally, the model predictions and feature importance are examined to support a more detailed comparison of classification performance.
[Insert Fig. 4 here: Overall DNA sequence classification workflow, showing dataset preparation, train-test split, k-mer and one-hot encoding pipelines, the four classification models, evaluation metrics, and additional SVM and XGBoost analyses.]

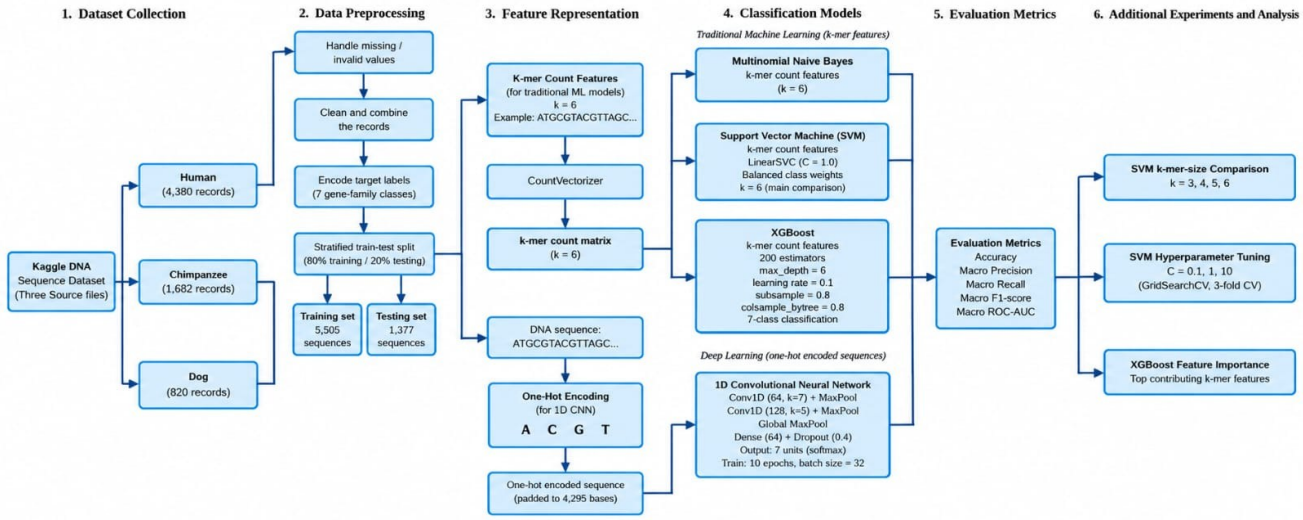


Fig. 1. Overall workflow of the DNA sequence classification and model evaluation pipeline.

G. Model Analysis

In addition to the overall evaluation metrics, further analyses were performed to examine model performance in more detail. The confusion matrix was used to examine the classification results of the main SVM model across the seven gene-family classes. It shows the numbers of correct predictions and misclassifications for each class. This analysis helps identify classes that are more frequently confused with other classes. Class-wise error analysis was also conducted for SVM to compare the number and proportion of misclassified sequences across the seven classes. This provides additional information about which classes were more difficult for the model to classify. For XGBoost, feature importance was examined to identify features that contributed to the trained model. This analysis provides information about the model's use of k-mer features, but feature importance alone does not establish a biological relationship or causal effect. These analyses complement the

main evaluation metrics by providing information about class-specific errors and the features used by the XGBoost classifier.

4.EXPERIMENTAL SETUP

This section describes the dataset, experimental configurations, model training procedures, and evaluation protocol used for DNA sequence classification. Four classification models were evaluated to compare traditional machine-learning and deep-learning approaches. Additional experiments were conducted to examine the effect of k-mer size, SVM hyperparameter tuning, and XGBoost feature importance.

A. Experimental Platform

The experiments were implemented using Python in a Kaggle notebook environment. The implementation included traditional machine-learning algorithms and a one-dimensional convolutional neural network (1D CNN). The same experimental dataset and training-test partition were used for the main model comparison.
The exact hardware specifications and software library versions were not recorded in the available experimental summary. Therefore, these details should be added from the original notebook environment if they are required for reproducibility.

B. Dataset

The experiments used the DNA Sequence Dataset obtained from Kaggle. The dataset contained three source files representing Human, Chimpanzee, and Dog sequences, with 4,380, 1,682, and 820 records, respectively. After combining the source files, the resulting dataset contained 6,882 sequences and three columns.
The classification task involved seven gene-family classes: G protein-coupled receptor, Tyrosine kinase, Tyrosine phosphatase, Synthetase, Synthase, Ion channel, and Transcription factor. The task was to predict the gene-family label of each DNA sequence rather than its species of origin.

C. Benchmark Configuration

Four classification models were evaluated: Multinomial Naive Bayes, Support Vector Machine (SVM), XGBoost, and 1D CNN.
For the traditional machine-learning models, DNA sequences were represented using k-mer count features generated by CountVectorizer, with a k-mer size of six for the main comparison. The SVM was implemented using LinearSVC with (C=1.0), balanced class weights, and a fixed random state. XGBoost was configured with 200 estimators, a maximum depth of 6, a learning rate of 0.1, a subsample ratio of 0.8, and a column subsampling ratio of 0.8.
For the deep-learning experiment, DNA sequences were converted into one-hot encoded representations and padded to a length of 4,295 bases. The 1D CNN consisted of two convolutional layers with 64 and 128 filters and kernel sizes of 7 and 5, respectively. The architecture also included max pooling, global max pooling, a dense layer with 64 units, dropout with a rate of 0.4, and a seven-unit softmax output layer. The model was trained for up to 10 epochs with a batch size of 32 and early stopping.

D. Experimental Design and Evaluation Protocol

The combined dataset was divided into training and testing sets using a stratified 80:20 split with random_state=42. The training set contained 5,505 sequences, while the testing set contained 1,377 sequences. Stratification was used to preserve the class distribution across the two sets.
The four models were evaluated on the same test partition for the main comparison. Model performance was measured using accuracy, macro-averaged precision, macro-averaged recall, macro F1-score, and macro ROC-AUC. These metrics were used to compare overall classification performance and performance across the seven classes.
Additional experiments were conducted using SVM to investigate the effect of k-mer sizes of 3, 4, 5, and 6. SVM hyperparameter tuning was performed using GridSearchCV with three-fold cross-validation and candidate values of (C=0.1), (1), and (10). XGBoost feature importance was also examined to identify features that contributed to its predictions. These analyses provided further insight into feature representation, model configuration, and classification performance.

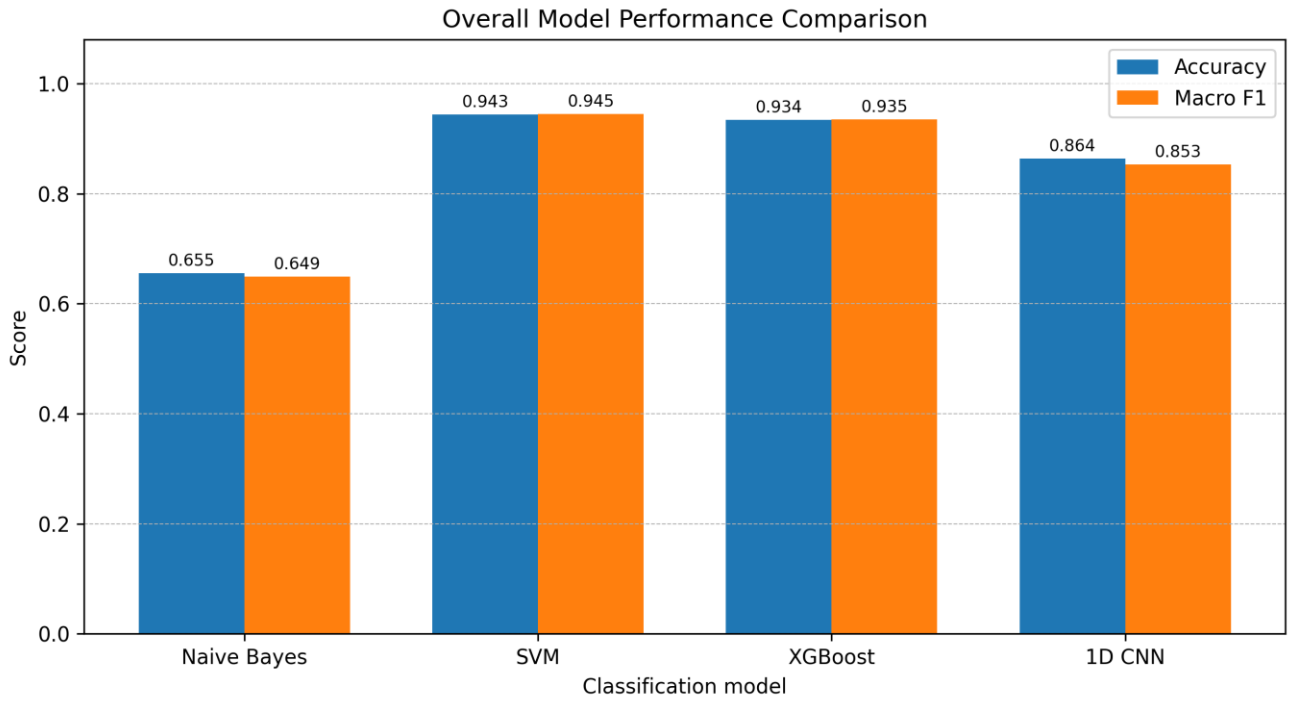
5. RESULTS AND DISCUSSION

This section presents the experimental results of four DNA sequence classification models: Multinomial Naive Bayes, Support Vector Machine (SVM), XGBoost, and a one-dimensional Convolutional Neural Network (1D CNN). The models were evaluated using accuracy, macro precision, macro recall, macro F1-score, and macro ROC-AUC. Additional experiments investigated the effect of k-mer size, SVM hyperparameter tuning, and class-wise classification errors.

A. Overall Classification Performance

Table I presents the performance of the four classifiers on the held-out test set. SVM achieved the highest accuracy of 94.34% and the highest macro F1-score of 94.46%. XGBoost achieved the highest macro precision of 95.33% and macro ROC-AUC of 0.9964. Multinomial Naive Bayes obtained the lowest accuracy and macro F1-score among the evaluated models.

	Accuracy	Precision	Recall	Macro F1	Macro ROC-AUC
SVM	0.9434	0.9476	0.9420	0.9446	0.9924
XGBoost	0.9339	0.9533	0.9198	0.9348	0.9964
1D CNN	0.8540	0.8643	0.8221	0.8371	0.9813
Multinomial Naive Bayes	0.6550	0.6701	0.6605	0.6489	0.8765



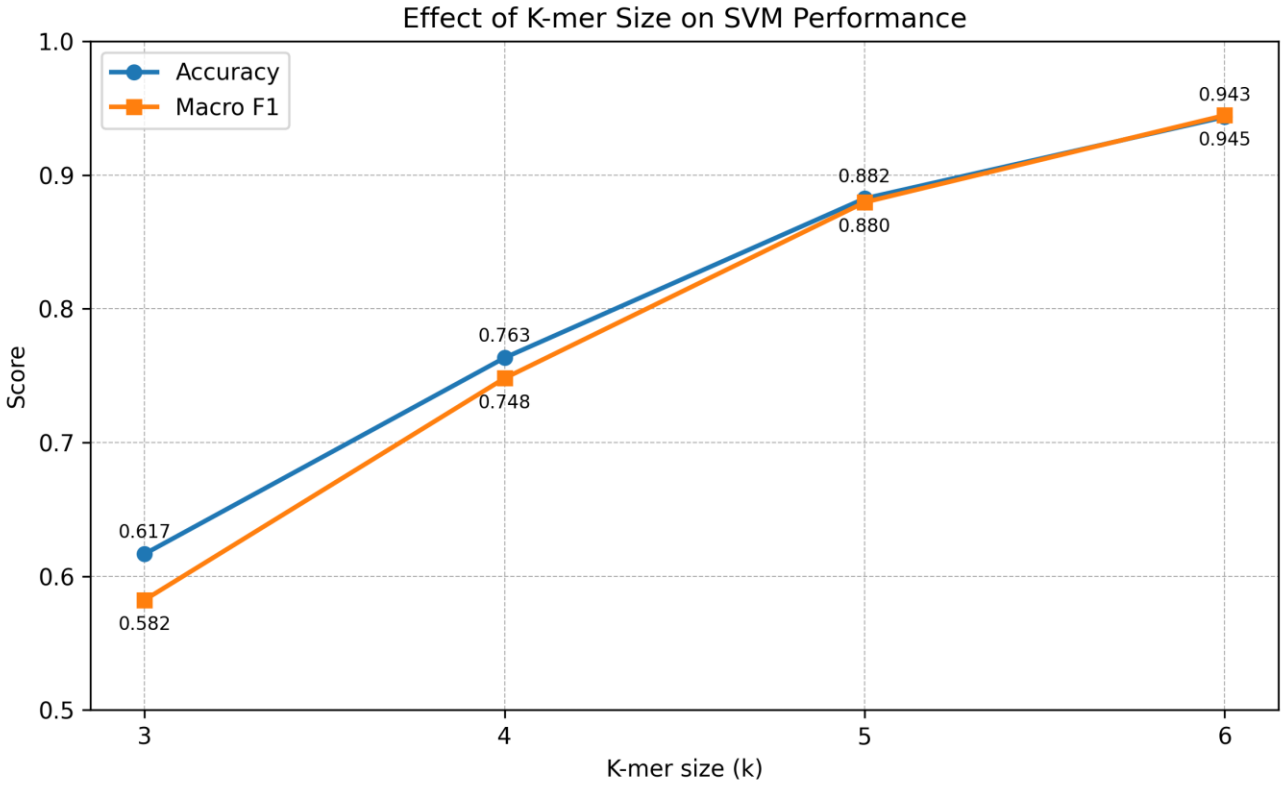
The results indicate that k-mer-based SVM was the strongest model in terms of accuracy and macro F1-score. XGBoost demonstrated strong discrimination according to macro ROC-AUC, although its accuracy and macro F1-score were slightly lower than those of SVM. The 1D CNN achieved better performance than Naive Bayes but did not outperform SVM or XGBoost on the reported test metrics. These findings suggest that traditional machine-learning models using k-mer count features provide strong baselines for this particular dataset. However, the results do not establish that traditional models will outperform deep-learning models on other DNA sequence datasets.

B. Effect of K-mer Size on SVM Performance

To investigate the effect of sequence feature representation, SVM was evaluated using k-mer sizes of 3, 4, 5, and 6. Table II summarizes the resulting classification performance.

```
Testing k-mer size: 6
```

	k	Accuracy	Precision	Recall	Macro F1	Training Time (sec)
3	6	0.9434	0.9476	0.9420	0.9446	17.1794
2	5	0.8824	0.8718	0.8893	0.8796	58.8753
1	4	0.7633	0.7349	0.7678	0.7480	12.0646
0	3	0.6166	0.5754	0.5939	0.5822	2.6571



SVM performance improved as the k-mer size increased from 3 to 6. Accuracy increased from 61.66% for three-mers to 94.34% for six-mers. Similarly, macro F1-score increased from 0.5822 to 0.9446. This trend suggests that six-mer count features captured more useful classification information under the experimental conditions used in this study.

However, the comparison does not prove that larger k-mer sizes will always perform better. In addition, a convergence warning occurred during the k = 5 experiment. The results for that setting should therefore be verified by rerunning the experiment with an adequate iteration limit.

C. SVM Hyperparameter Tuning Analysis

GridSearchCV was used to evaluate SVM regularization parameter values of (C = 0.1), (1), and (10) using three-fold cross-validation on the training data. The best cross-validation configuration was (C = 0.1), with a macro F1-score of 0.8983.

In the subsequent test evaluation, the tuned SVM achieved 94.34% accuracy and a macro F1-score of 0.9427. The original SVM achieved the same accuracy and a slightly higher macro F1-score of 0.9446. Therefore, hyperparameter tuning did not improve the reported test performance.

Convergence warnings occurred during some cross-validation fits, which may affect the reliability of the tuning results. Further experiments with appropriate convergence settings and training-only model selection are needed to confirm the best configuration.

```
Best parameters: {'C': 0.1}
Best cross-validation Macro F1: 0.8983
```

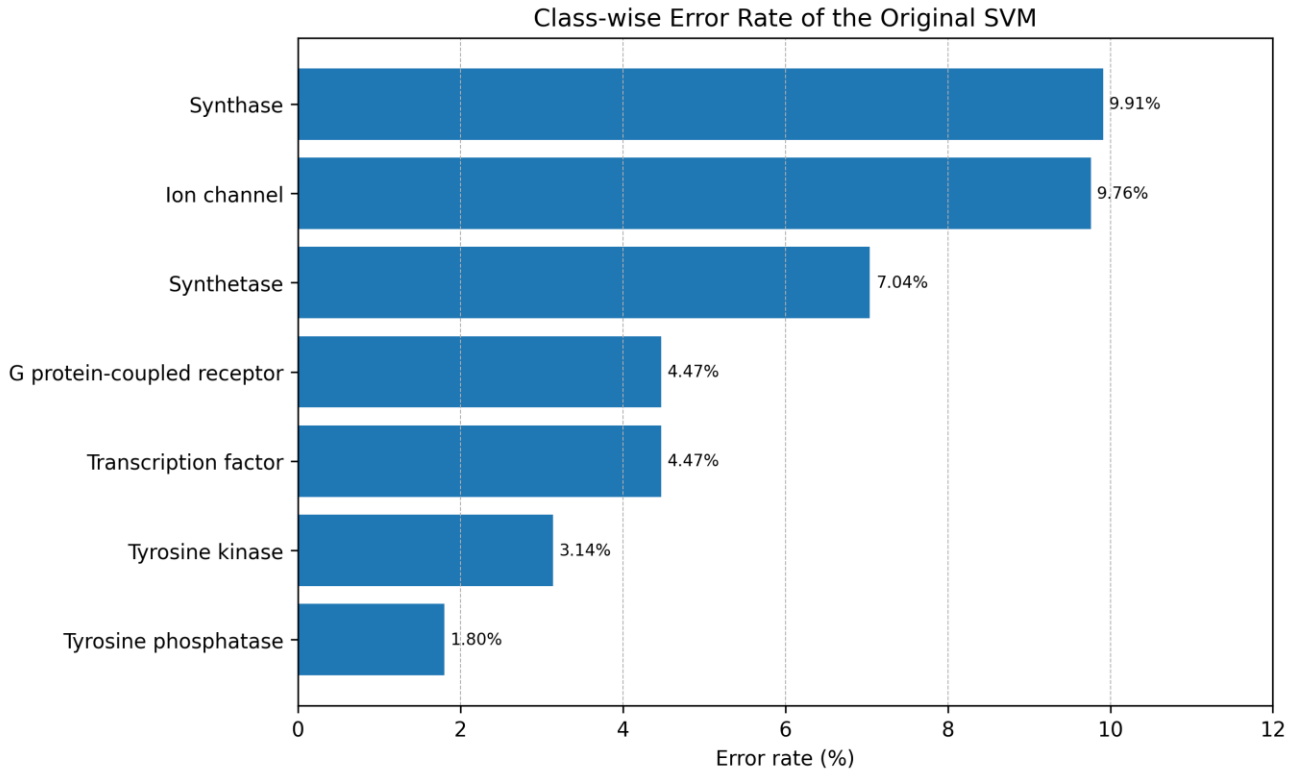
D. Class-wise Classification and Error Analysis

Class-wise error analysis was conducted to identify the gene-family classes that were more challenging for the original SVM model. Table III reports the number and proportion of incorrectly classified examples in each class.

Table III. Class-wise SVM classification errors

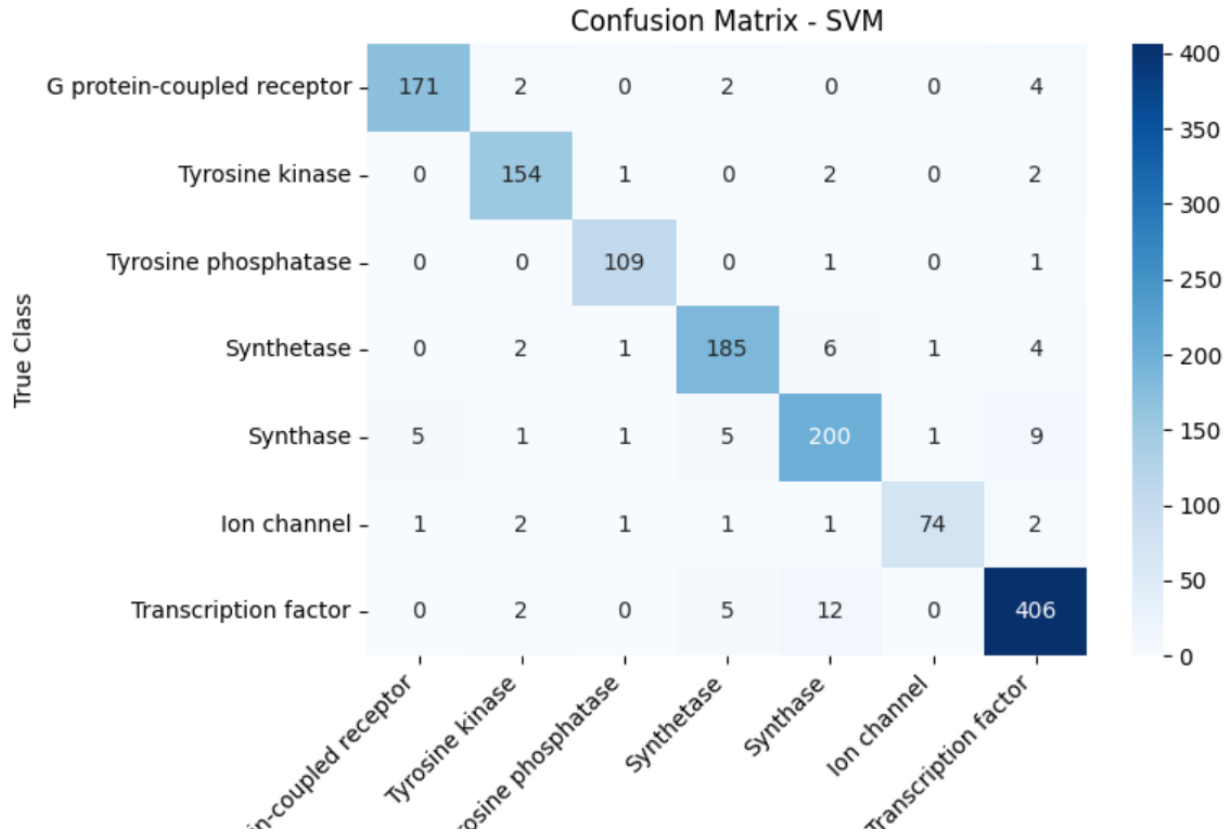
Gene-family class	Test examples	Errors	Error rate
Synthase	222	22	9.91%
Ion channel	82	8	9.76%
Synthetase	199	14	7.04%
Transcription factor	425	19	4.47%
G protein-coupled receptor	179	8	4.47%
Tyrosine kinase	159	5	3.14%

Tyrosine phosphatase	111	2	1.80%



Synthase had the highest error rate at 9.91%, followed closely by Ion channel at 9.76%. Synthetase also showed a comparatively higher error rate of 7.04%. In contrast, Tyrosine phosphatase had the lowest error rate at 1.80%.

These differences indicate that the SVM did not classify all gene-family classes equally well. The errors may be associated with overlapping sequence patterns or differences in the representation of the classes, but these possible explanations require further investigation. The confusion matrix can provide additional information about which pairs of classes were most frequently confused.



E. CNN Performance and XGBoost Feature Importance

The 1D CNN achieved 86.35% accuracy, 87.08% macro precision, 84.40% macro recall, and a macro F1-score of 0.8527. Its macro ROC-AUC was 0.9806. Although the CNN demonstrated useful classification performance, it did not outperform the SVM or XGBoost models in the main comparison.

The CNN used one-hot encoded sequences padded to 4,295 bases and was trained for up to 10 epochs with a batch size of 32 and early stopping. The difference in performance may be related to the dataset size, sequence representation, model configuration, or training procedure. However, the present experiment does not establish which factor caused the difference.

XGBoost feature importance was also examined to identify k-mer features that contributed to its predictions. These features may help describe which sequence patterns the model relied on. However, feature importance alone does not demonstrate biological function, causal relationships, or biological significance.

F. Overall Discussion and Limitations

The experiments showed that SVM achieved the best accuracy and macro F1-score, while XGBoost achieved the highest macro ROC-AUC. The k-mer experiment further demonstrated that six-mer representation produced the strongest SVM results among the four tested k-mer sizes. Hyperparameter tuning did not improve the original SVM's reported macro F1-score, and the CNN did not surpass the best traditional machine-learning models under the selected configuration.

Several limitations should be considered when interpreting these findings. First, the experiments used a single dataset and a single stratified train-test split. Second, the test set was reused for multiple comparisons, including the k-mer experiments, so it was not an entirely untouched evaluation set

throughout the study. Third, no external dataset was used to assess generalization. Fourth, convergence warnings occurred in some SVM experiments. Finally, the CNN's fixed sequence length and training configuration may have influenced its performance.
Future work should include training-only cross-validation for model selection, a final evaluation on an untouched test set, external validation on an independent dataset, and further investigation of class-specific errors. These steps would provide stronger evidence of the reliability and generalizability of the proposed classification approach.

6. CONCLUSION AND FUTURE WORK

This study compared four classification models—Multinomial Naive Bayes, Support Vector Machine (SVM), XGBoost, and a one-dimensional Convolutional Neural Network (1D CNN)—for seven-class DNA sequence classification. The experiments used 6,882 records from the combined Human, Chimpanzee, and Dog files. The traditional machine-learning models used six-mer count features, whereas the CNN used one-hot-encoded DNA sequences.

Among the evaluated models, SVM achieved the highest accuracy of 94.34% and the highest Macro F1-score of 94.46%. XGBoost achieved the highest Macro ROC-AUC of 99.64%. The SVM k-mer experiments also showed that six-mer features outperformed the tested three-mer, four-mer, and five-mer configurations. These findings indicate that k-mer-based traditional machine-learning models can provide strong classification performance on the selected dataset.

Despite these promising results, the study has limitations, including the use of a single dataset and train-test split, the absence of external validation, and the need for further investigation of class-specific errors. Future work will focus on repeated stratified cross-validation, evaluation using an untouched test set, and testing on independent datasets. Additional model configurations and k-mer sizes may also be investigated. Furthermore, the biological relevance of important k-mers should be examined using domain knowledge or external biological annotations rather than inferred from feature importance alone.

In conclusion, this study provides an initial comparative benchmark of traditional machine-learning and deep-learning approaches for DNA sequence classification on the selected dataset. Further validation and biological investigation are needed to assess the generalizability and practical significance of the findings.

REFERENCES

[1] N. Singh, "DNA Sequence Dataset," Kaggle. [Online]. link: **https://www.kaggle.com/datasets/nageshsingh/dna-sequence-dataset**. Accessed: Oct. 19, 2026.

[2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794. doi: **10.1145/2939672.2939785**.

[3] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, pp. 273–297, 1995. doi: **10.1007/BF00994018**.

[4] A. McCallum and K. Nigam, "A comparison of event models for Naive Bayes text classification," in *Proc. AAAI-98 Workshop on Learning for Text Categorization*, 1998, pp. 41–48. **[Online]. Available: https://aaai.org/papers/041-ws98-05-007/**.

[5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. doi: **10.1109/5.726791**.

[6] Scikit-learn developers, "Scikit-learn User Guide." [Online]. Available: **https://scikit-learn.org/stable/user_guide.html**. Accessed: Oct. 19, 2026.

[7] H. Gunasekaran, K. Ramalakshmi, A. R. M. Arokiaraj, S. D. Kanmani, C. Venkatesan, and C. S. G. Dhas, "Analysis of DNA sequence classification using CNN and hybrid models," *Computational and Mathematical Methods in Medicine*, vol. 2021, Art. no. 1835056, 2021. doi: **10.1155/2021/1835056**.

[8] S. Juneja, A. Dhankhar, A. Juneja, and S. Bali, "An approach to DNA sequence classification through machine learning," *International Journal of Reliable and Quality E-Healthcare*, vol. 11, no. 2, 2022. doi: **10.4018/IJRQEH.299963**.

[9] C. Ao, S. Jiao, Y. Wang, L. Yu, and Q. Zou, "Biological sequence classification: A review on data and general methods," *Research*, vol. 2022, Art. no. 0011, 2022. doi: **10.34133/research.0011**.